Databricks Asset Bundles (DABs)

The Complete End-to-End, Step-by-Step Guide

From zero to deploying notebooks, workflow jobs and Lakeflow (Delta Live Tables) pipelines to DEV and PROD using the Databricks CLI, VS Code, Databricks Git folders and the Web Terminal

Built from Ansh Lamba's 3-hour masterclass
youtu.be/Yz0pTr7LYG4

Covers: Free Edition setup, CLI install and authentication, project initialisation, databricks.yml, jobs, presets, variables, pipelines, change tracking, DEV and PROD targets, Git folders, destroy, and interview preparation

Table of Contents

Part 0: How to Use This Guide **7**

0.1 What you will be able to do at the end . . . 7
0.2 What are Databricks Asset Bundles? . . . 7
0.3 Prerequisites . . . 8
0.4 The plan of the video in one table . . . 8

Part 1: Create Your Free Databricks Account **9**

1.1 Sign up for Free Edition . . . 9
1.2 If you are new to Databricks . . . 9

Part 2: Install the Tools **10**

2.1 Install VS Code and Python . . . 10
2.2 Install uv (optional but recommended) . . . 10
2.3 Install the Databricks CLI . . . 10
2.4 Verify the installation . . . 11

Part 3: Understanding the Databricks CLI **12**

3.1 What is the CLI? . . . 12
3.2 How does it work behind the scenes? . . . 12
3.3 Getting help from the CLI itself . . . 12

Part 4: Authenticate the CLI with Your Workspace **13**

4.1 The two authentication types . . . 13
4.2 Find your workspace host URL . . . 13
4.3 Log in and create the dev profile . . . 13
4.4 Where is the profile saved? The .databrickscfg file . . . 14
4.5 List your profiles . . . 14
4.6 Why always pass the profile . . . 14

Part 5: Practise with CLI Commands **15**

5.1 List catalogs . . . 15

5.2 Try to create a catalog from the CLI (expected to fail) . . . 15
5.3 Inspect the catalog with the CLI . . . 15
5.4 Create a schema from the CLI . . . 16

Part 6: CI/CD and Where Bundles Fit **17**

6.1 The DevOps loop . . . 17
6.2 The official definition . . . 17
6.3 What a bundle can contain . . . 17
6.4 When should you use bundles? . . . 17
6.5 Who does what in a real organisation . . . 18

Part 7: Local Development - Initialise a Bundle **19**

7.1 Create an empty working folder and open it . . . 19
7.2 Initialise the bundle . . . 19
7.3 Re-open VS Code inside the bundle folder . . . 20
7.4 Tour of the folder structure . . . 20
7.5 Clean the sample content . . . 21
7.6 Create a PROD folder in the workspace . . . 21

Part 8: Your First Deployment (Notebook to DEV) **22**

8.1 Create a notebook . . . 22
8.2 Understand databricks.yml . . . 22
8.3 Look at the bundle commands . . . 23
8.4 Deploy . . . 23
8.5 Verify in the workspace . . . 23

Part 9: Add a Workflow Job to the Bundle **25**

9.1 Create the job in the UI first (recommended) . . . 25
9.2 Put the YAML in the bundle (the right way) . . . 25
9.3 Use relative paths (very important) . . . 26
9.4 Deploy and fix the first error . . . 26
9.5 Verify . . . 27

Part 10: Presets - Prefixes and Source-Linked Deployment **28**

10.1 Why does the job name have a prefix? 28

10.2 The default formula 28

10.3 Add your own custom presets 28

10.4 What is source_linked_deployment? 29

Part 11: How Bundles Track Changes (.databricks) **30**

11.1 The sync snapshot 30

11.2 Prove it: add and change notebooks 30

11.3 The trap: deleting something directly in the workspace 30

11.4 The fix 31

Part 12: Deploy to PROD **32**

12.1 Add the prod target 32

12.2 Why you did not need a profile in bundle commands 32

12.3 Validate and summarise before deploying 33

12.4 Deploy to PROD 33

12.5 Verify 33

Part 13: Git Folders and the Web Terminal **34**

13.1 Background 34

13.2 Create a GitHub repository 34

13.3 Link GitHub to Databricks 34

13.4 Create the Git folder 34

13.5 Create a feature branch 35

13.6 Open the Web Terminal 35

Part 14: Initialise a Bundle in the Git Folder **36**

14.1 Move to the right directory 36

14.2 Clean and configure 36

Part 15: Variables and Dynamic Deployment **37**

15.1 Write a parameterised notebook 37

15.2 Create the job in the UI . . . 37
15.3 Copy the YAML into a resource file . . . 38
15.4 Create the variable . . . 38
15.5 Refer to the variable . . . 38
15.6 Make sure include is present . . . 39

Part 16: Add a Lakeflow (Delta Live Tables) Pipeline 40

16.1 Turn on the new editor . . . 40
16.2 Create a demo pipeline . . . 40
16.3 Move the pipeline root into your bundle . . . 41
16.4 Copy the pipeline YAML . . . 41
16.5 Fix the paths and the catalog . . . 41
16.6 Set the targets with presets . . . 42
16.7 Validate, summarise and deploy to DEV (Web Terminal) . . . 42
16.8 Verify . . . 43

Part 17: Override Variables at Deploy Time 44

17.1 Find the flag . . . 44
17.2 Deploy to PROD with an overridden catalog . . . 44
17.3 Verify PROD . . . 44

Part 18: Destroy a Bundle 46

18.1 What destroy does . . . 46
18.2 Run it . . . 46
18.3 Why destroy is useful . . . 46

Part 19: Push Changes to GitHub and Merge 47

Part 20: Command Cheat Sheet 48

Variable and path syntax . . . 48

Part 21: Troubleshooting Guide 49

Part 22: Interview Questions and Answers 50

Part 23: Final Checklist and Key Takeaways **53**

23.1 Build checklist 53

23.2 Ten things to remember 53

Part 0: How to Use This Guide

This guide turns the whole video into a build you can follow on your own machine. Every part follows the same order as the video, so you can watch and build side by side, or just read and build. Follow the steps in order. Each part ends with a short checkpoint so you know you are on track before moving on.

0.1 What you will be able to do at the end

- Install and authenticate the Databricks CLI, which is the backbone of asset bundles.
- Initialise a bundle project and explain every folder and file inside it.
- Deploy notebooks and source code files to a Databricks workspace.
- Add Databricks Workflow jobs to a bundle using relative paths.
- Add Lakeflow Declarative (Delta Live Tables) pipelines using the new pipeline editor.
- Use presets and custom variables to make one bundle deploy to many environments.
- Understand how the hidden .databricks folder tracks changes between deployments.
- Deploy to DEV and PROD, override variables at deploy time, and destroy a bundle.
- Do all of the above from VS Code (local) and from Databricks Git folders with the Web Terminal.
- Answer asset bundle interview questions with confidence.

0.2 What are Databricks Asset Bundles?

Databricks Asset Bundles, shortened to **DABs**, are the modern way to deploy your changes to different environments such as DEV, QA and PROD. A bundle is an **end-to-end definition of a project**: how it is structured, tested and deployed. It keeps your source files (notebooks, Python files) and your metadata (jobs, pipelines, targets, settings) together as one unit, written as code in YAML, and deploys them as a single bundle to a target environment.

> **Why it matters:** Asset bundles are not only about bundles. They bring together DevOps, CI/CD, local development, Git inside Databricks, and the Databricks CLI. That is exactly why interviewers love this topic: if you can say you have deployed bundles, and explain how, your answer carries real weight.

0.3 Prerequisites

Requirement	Details
Databricks knowledge	Very basic only: workspace, notebooks, jobs. You do not need to be an expert.
Databricks account	Free Edition is enough. No cloud account (Azure, AWS or GCP) and no payment needed.
VS Code	Installed on your computer (used for local development).
Python	Installed on your computer.
uv (recommended)	A very fast Python package manager written in Rust, 10 to 100 times faster than pip.
GitHub account and repo	Needed only for Part 13 (Git folders). Takes about 5 minutes to create.
Notebook for notes	Take notes as you go. The video repeats this many times.

0.4 The plan of the video in one table

Phase	What you do
1. Setup	Create the Free Edition account, install the CLI, authenticate with a profile.
2. CLI practice	List catalogs, create a schema using only CLI commands.
3. Local bundle (VS Code)	Initialise, deploy notebooks, add a job, presets, change tracking, deploy to PROD.
4. Git folders (Databricks UI)	Clone a repo, use the Web Terminal, add variables, a job and a pipeline.
5. Finish	Override variables, destroy a bundle, push to GitHub, merge, pull.

Part 1: Create Your Free Databricks Account

Asset bundles are normally shown with a cloud provider, but this guide uses the **Databricks Free Edition** so you spend nothing and still learn everything, including deployment.

1.1 Sign up for Free Edition

Step 1. Open any browser and search for **Databricks Free Edition**.

Step 2. Click **Try for free**. The page you land on offers the express setup called *Databricks for Work* (a 14-day free trial with up to 400 dollars in credits). **Do not use that one**, because it expires after 14 days.

Step 3. Look for the link that says you want Databricks for personal use, with core features free forever with Free Edition. The video notes this link was moved by Databricks, so read the page carefully. Click it.

Step 4. Click **Get started for free** (not Databricks for Work).

Step 5. Sign up with Google or Microsoft or any email. You do not need a business email.

Step 6. Once the account exists, go to the login page and use the **already using Databricks, log in** link. You land in your workspace.

> **Note:** Free Edition is almost identical to the paid cloud version, with some restrictions such as limited control at the metastore level. You will meet one of these restrictions later (creating a catalog from the CLI fails). That is expected and the workaround is simple.

1.2 If you are new to Databricks

Ansh Lamba's channel has a video called *Modern Databricks with Unity Catalog* that covers workspace basics, Unity Catalog, metastore and external locations. Watching it is optional but helpful, because asset bundles is a separate topic linked to CI/CD and deployment, and only needs light Databricks knowledge.

> **Tip:** Checkpoint: you can log in to a Databricks workspace and see the left sidebar with Workspace, Catalog, Compute and Jobs and Pipelines.

Part 2: Install the Tools

2.1 Install VS Code and Python

Install Visual Studio Code and Python from their official websites if you do not have them. The video assumes both are installed.

2.2 Install uv (optional but recommended)

uv is a modern Python package manager (built by Astral). Many organisations are moving from pip to uv because of its speed. You can install it with pip.

Step 1. Open Command Prompt (cmd).

Step 2. Run the command below.

```
pip install uv
```

You can also install uv by other methods, for example a PowerShell script on Windows, Homebrew, or pipx. The video prefers pip when it is already available.

2.3 Install the Databricks CLI

The Databricks CLI (Command Line Interface) is software, not just a Python library. That is why it is installed with winget (Windows package manager) and not pip.

Windows (the method used in the video)

Step 1. Open Command Prompt.

Step 2. Optionally search first to confirm the package name:

```
winget search databricks
```

Step 3. Install the CLI:

```
winget install Databricks.DatabricksCLI
```

Step 4. If you already have it, winget says *Found an existing package already installed* and tries to upgrade it. That is fine. Running the command many times is safe.

Step 5. The CLI is downloaded from the official Databricks GitHub repository.

macOS and Linux

On the official Databricks CLI install page, use the Homebrew or curl commands shown for your operating system. Windows users can also use Chocolatey or WSL. The page is under the Databricks documentation: CLI, then Install or update.

2.4 Verify the installation

```
databricks -v
```

You should see a version such as *Databricks CLI v0.268.0* (the version in the video). Yours may be newer, which is fine.

> **Warning:** If the command is not recognised: close all terminals and open a new one, then try again. Wait 5 to 10 minutes. If it still fails, restart your laptop. Software installed through winget takes time to reflect in the system PATH. Do the same restart check for VS Code if the VS Code terminal does not recognise it.

> **Tip:** Checkpoint: databricks -v prints a version, both in Command Prompt and in the VS Code terminal.

Part 3: Understanding the Databricks CLI

Before bundles, you need a clear picture of what the CLI is, because **the bundle commands are built entirely on top of it**. You cannot even initialise a bundle without the CLI.

3.1 What is the CLI?

The Databricks CLI lets you interact with the Databricks platform from your local terminal or from automation scripts. You can also run CLI commands inside a Databricks workspace using the **Web Terminal**, which is useful when an organisation does not allow a local terminal.

3.2 How does it work behind the scenes?

The CLI is a wrapper built on top of the Databricks REST APIs. When you run a command such as listing catalogs, the CLI sends the REST API request for you and shows the result, so you do not have to build the API calls by hand.

You type	Behind the scenes
databricks catalogs list	A REST API call to list Unity Catalog catalogs.
databricks schemas create ...	A REST API call (POST) that creates a schema.

You are not expected to memorise commands. Use the documentation or the built-in help flag whenever you need syntax.

3.3 Getting help from the CLI itself

```
databricks -h               # list every available command
databricks catalogs -h      # help for one command group
databricks catalogs create -h   # help for one specific command
```

Help works hierarchically. Go from the group to the sub-command and read the *Usage* section to see the exact syntax, the available commands (get, list, update, delete, create) and the flags.

> **Tip:** Do not memorise commands. Understand the pattern: databricks, then the command group, then the sub-command, then arguments, then flags, and always add -h when unsure.

Part 4: Authenticate the CLI with Your Workspace

Databricks must know that the requests coming from your CLI are genuine. That is what authentication does.

4.1 The two authentication types

Type	Meaning	When to use
U2M	User to Machine. You log in as yourself through the browser.	Easiest to set up. Most common for learning and for developers in organisations.
M2M	Machine to Machine. Uses service principals.	Ideal for automated pipelines in organisations with a cloud provider.

This guide uses **U2M**.

4.2 Find your workspace host URL

Step 1. In Databricks, open **Compute**, then **SQL Warehouses**, then click the serverless warehouse.

Step 2. Open **Connection details** and copy the **Server hostname**. Alternatively copy the URL from your browser address bar.

Step 3. Add **https://** in front of it. Always use the full URL starting with https://, otherwise you can get errors later when deploying the bundle.

4.3 Log in and create the dev profile

Step 1. Open a terminal (VS Code terminal is fine).

Step 2. Run the login command with your host.

```
databricks auth login --host https://<your-workspace-url>
```

Step 3. When it asks for a **profile name**, type **dev** and press Enter. This profile represents your development environment.

Step 4. A browser window opens. Approve the login. The terminal then says the profile was successfully saved.

4.4 Where is the profile saved? The .databrickscfg file

The CLI created a configuration file named **.databrickscfg**. On Windows it sits in your user folder, for example C:\Users\<your-name>. It stores every profile you create. It looks like this:

```
[DEFAULT]

[dev]
host      = https://<your-workspace-url>
auth_type = databricks-cli
```

- The **[DEFAULT]** section is a fallback. It is used when you run a command without specifying a profile.
- If you leave DEFAULT empty or remove it, you simply must give the profile name every time. Nothing breaks.
- **[dev]** is the profile you just created: a name, the host, and the auth type (databricks-cli).

4.5 List your profiles

```
databricks auth profiles
```

You will see each profile name, its host URL and whether it is valid. The video found this command by asking an AI assistant, which shows it is fine to look commands up.

4.6 Why always pass the profile

If you have only one profile, commands work without it. But with several profiles the CLI gets confused about which one to use. So the good habit is to always add **--profile dev** (or -p dev) in commands.

> **Note:** Later, for bundle commands, you will not need to give the profile. The bundle reads the host from your target and looks it up in .databrickscfg automatically. This is explained in Part 12.

> **Tip:** Checkpoint: databricks auth profiles shows your dev profile as valid.

Part 5: Practise with CLI Commands

This short practice proves your connection works and shows that the CLI can create real objects. It also prepares the catalog and schema used later.

5.1 List catalogs

Step 1. Run:

```
databricks catalogs list --profile dev
```

Step 2. In a fresh Free Edition workspace you will see the default catalogs such as **system** and a managed workspace catalog. The video also had an older catalog created earlier, yours will not.

5.2 Try to create a catalog from the CLI (expected to fail)

```
databricks catalogs create asset_bundles --profile dev
```

On Free Edition this returns an error saying the metastore root URL does not exist and that default storage is enabled, so use the UI to create a new catalog. This is not a mistake of yours. You do not own the metastore in Free Edition, so you cannot create catalogs this way.

Workaround: create the catalog in the UI

Step 1. Go to **Catalog** in the workspace and click **Create catalog**.

Step 2. Name it **asset_bundles** (the video says asset bundles; Unity Catalog names cannot contain spaces, so use an underscore).

Step 3. Keep the default options, grant workspace access as shown, and save.

5.3 Inspect the catalog with the CLI

```
databricks catalogs get asset_bundles --profile dev
```

You get catalog details: name, type (managed), creator, metastore ID, storage location (owned by Databricks in Free Edition), and more.

5.4 Create a schema from the CLI

Step 1. Ask for help to learn the syntax:

```
databricks schemas create -h
```

Step 2. The usage is: schema name first, then catalog name. Create the bronze schema:

```
databricks schemas create bronze asset_bundles --profile dev
```

Step 3. Refresh Catalog in the UI. Open asset_bundles. You will see the **bronze** schema, created purely by a CLI command.

> **Why it matters:** Why this exercise matters: asset bundle commands are purely built on the CLI. You need the CLI even to initialise a bundle. After this, bundle commands will feel familiar.

Part 6: CI/CD and Where Bundles Fit

6.1 The DevOps loop

DevOps is an endless loop. Work is planned, the code base is structured, code is built and tested, a release is made, it is deployed to environments, finally made live in PROD, then monitored. When enhancements are needed, you plan again and the loop continues.

Side of the loop	Meaning
Left half (Dev)	Plan, code, build, test. This is Continuous Integration (CI).
Right half (Ops)	Release, deploy, operate, monitor. This is Continuous Deployment or Delivery (CD).

Asset bundles belong to the right part: deployment. They give you a new, efficient and easily configurable way to deploy changes to QA, PROD and other environments.

6.2 The official definition

Databricks asset bundles are a tool to facilitate adopting software development lifecycle (SDLC) practices. A bundle includes metadata alongside project source files, so jobs and pipelines are described as source. Ultimately a bundle is an **end-to-end definition of a project** covering structure, testing and deployment. Everything is deployed together as a single bundle to a target environment.

6.3 What a bundle can contain

- Required infrastructure and settings for Databricks resources.
- Source files such as notebooks and Python files.
- Definitions for jobs and Lakeflow (Delta Live Tables) pipelines.
- Unit tests and integration tests.

6.4 When should you use bundles?

Use them for data analytics and ML projects in a team-based environment, so source files are organised and managed consistently, collaboration is smooth, and deployments are repeatable. Requirements: the Databricks CLI installed, and a Databricks runtime version if you are not using serverless.

6.5 Who does what in a real organisation

A DevOps or CI/CD engineer usually automates the deployment to QA and PROD. But the **data engineer builds the bundle**: the folder structure and the YAML files. The DevOps engineer takes your tested bundle and automates it. So you must learn bundles even if someone else automates the pipeline.

> **Interview:** Interview angle: be ready to explain that a bundle is a project defined as code, deployed through the CLI to targets such as dev and prod, and that the data engineer authors it.

Part 7: Local Development - Initialise a Bundle

7.1 Create an empty working folder and open it

Step 1. Create an empty folder on your computer, for example **asset_bundles_tutorial**.

Step 2. In VS Code choose **File, Open Folder** and pick it. It should be empty, because you are developing something new.

Step 3. Open the terminal: the three dots menu, then Terminal, then New Terminal. The keyboard shortcut is **Ctrl + ` (backtick)**.

Step 4. Run **databricks -v** to confirm VS Code can see the CLI. If not, restart VS Code or the laptop.

7.2 Initialise the bundle

Step 1. Run:

```
databricks bundle init
```

Step 2. The CLI asks which template to use. Press Enter to accept the default **default-python** template.

Step 3. Answer the prompts:

Prompt	Answer in the video
Unique name for this project	dab_tutorial (the video says dab tutorial; use underscores)
Include a stub (sample) notebook	yes
Include a stub Lakeflow (DLT) pipeline	yes
Include a stub Python package	yes
Use serverless compute	yes (the only option on Free Edition)

The bundle folder is created inside your current folder. You answered yes to every sample option so you can study the standard structure. Later you will delete the sample files and keep the folders.

7.3 Re-open VS Code inside the bundle folder

Use **File, Open Folder** and select the new **dab_tutorial** folder itself. Now you are inside the bundle project. Terminal commands run in the right place without needing cd every time.

7.4 Tour of the folder structure

Item	Purpose
.databricks	Hidden folder that works as the transaction log (like .git or Delta log). It stores deployment snapshots. Covered in Part 11.
.vscode	VS Code settings only. You can ignore it.
fixtures	Test data and fixtures. Contains only a .gitkeep file so the empty folder is tracked by Git.
resources	Where you put YAML definitions of jobs, pipelines and variables (resources).
scratch	Scratchpad for quick analysis and prototypes. Listed in .gitignore, so it is not tracked or deployed.
source (src)	The most important folder. Holds your source code: notebooks, scripts, pipeline code.
tests	Test files (unit and integration tests).
databricks.yml	The main bundle file: the backbone of the bundle.
.gitignore	Files and folders Git must ignore, such as scratch.
README and project files	Documentation and packaging files from the template.

> **Note:** Your folder names can differ from the video depending on the template and your answers. You are not forced to use these exact names. The video author likes this structure and keeps it, then deletes the sample content. Follow whichever structure you prefer, but keep the folder names consistent with your YAML paths.

7.5 Clean the sample content

Step 1. Delete the sample files inside **resources**, **source** and **tests** (two sample test files).

Step 2. Keep the empty folders as the skeleton. Keep the scratch folder too.

Step 3. Open **databricks.yml** and delete the *artifacts* section (not needed, you are not building wheels) and, for now, the *include* section.

7.6 Create a PROD folder in the workspace

For this local demo you will simulate a production environment in the same workspace.

Step 1. In Databricks, open **Workspace, Home**.

Step 2. Remove any sample files if you want a clean view.

Step 3. Create a new folder named **prod**. Leave it as it is.

> **Tip:** Checkpoint: VS Code is opened inside the bundle folder, the structure is empty, and a prod folder exists in your workspace home.

Part 8: Your First Deployment (Notebook to DEV)

8.1 Create a notebook

Step 1. Inside **source**, create a folder named **notebooks**.

Step 2. Inside it create a file **notebook_1.ipynb** (.ipynb is the IPython notebook extension).

Step 3. Add one code cell: **print('hello world')**.

You are only writing code. Do not run Spark code locally, because no cluster is attached to VS Code. This is development, not execution.

Think of a real project: in Sprint 1 you may only finish one or two notebooks. You do not deploy every job and pipeline in the world. You simply deploy what you built. That is what a bundle lets you do.

8.2 Understand databricks.yml

YAML stands for *YAML Ain't Markup Language* (originally Yet Another Markup Language). It is a human-readable alternative to JSON, which becomes messy when deeply nested. DevOps and CI/CD tools prefer YAML for configuration.

YAML rules you need

- Every entry is a key and a value, written as **key: value**. Strings do not need double quotes.
- Indentation (spaces, never tabs) shows hierarchy.
- A list uses a hyphen for each item.

```
my_list:
  - 1
  - 2
  - 3
```

A minimal databricks.yml for DEV:

```
bundle:
  name: dab_tutorial

targets:
  dev:
    mode: development
    default: true
    workspace:
      host: https://<your-workspace-url>
```

Key	Meaning
bundle.name	The bundle (project) name. Should match your project name.
uuid	A unique identifier created by the template. No need to worry about it.
include	Which other YAML files to load. Explained in Part 9.
targets	The environments you can deploy to: dev, qa, prod. Each target has its own settings.
mode: development	Marks this target as development and adds default behaviour (a prefix on resource names).
default: true	This target is used when you do not pass a target.
workspace.host	The workspace URL for this target.

8.3 Look at the bundle commands

```
databricks bundle -h
```

Important commands: **validate**, **deploy**, **summary**, **run**, **generate**, **init**, **destroy**, **deployment**. You already used init.

8.4 Deploy

Step 1. Make sure your terminal is inside the bundle folder.

Step 2. Run:

```
databricks bundle deploy -t dev
```

Step 3. The CLI prints that it is building and uploading files and then finishes.

8.5 Verify in the workspace

Step 1. In Databricks open **Workspace, Home**.

Step 2. You will see a new hidden-style folder **.bundle**. Open it, then **dab_tutorial**, then **dev**, then **files**.

Step 3. You will see your project files: source, fixtures and so on. Open **source, notebooks, notebook_1** to see the same code you wrote.

The **resources** folder is missing. Empty folders are not deployed. Folders appear after they contain something.

Note: The workspace path pattern is Home / .bundle / bundle name / target / files. Remember it. The video author explains later that this location can be changed for PROD with root_path.

Tip: Checkpoint: your notebook is visible in Databricks under .bundle/dab_tutorial/dev/files/source/notebooks.

Part 9: Add a Workflow Job to the Bundle

In real projects, after notebooks you create jobs to orchestrate them. Deploying jobs through a bundle is the realistic scenario.

9.1 Create the job in the UI first (recommended)

Writing job YAML by hand and getting it fully right is very hard. The recommended approach: build the job in the Databricks UI, then copy its YAML.

Step 1. Go to **Jobs and Pipelines**, then **Create, Job**.

Step 2. Add a task, name it for example **ingest**, type **Notebook**.

Step 3. Pick the notebook path from **.bundle/dab_tutorial/dev/files/source/notebooks/notebook_1** and confirm.

Step 4. Compute: **Serverless**. Click **Create task**.

Step 5. Rename the job to **demo job** (Edit job, click the name).

Step 6. Click the three dots next to Run now and choose **Edit as YAML**. Copy the YAML.

9.2 Put the YAML in the bundle (the right way)

You could paste the YAML into databricks.yml, but with 20 jobs and 30 pipelines that file becomes a mess. databricks.yml is the parent file for targets and settings. Keep each resource in its own file.

Step 1. In **resources**, create a folder named **jobs**.

Step 2. Create the file **resources/jobs/demo_job.yml** and paste the YAML copied from the UI.

Step 3. Add this to **databricks.yml** so the parent file loads all resource files:

```
include:
  - resources/*.yml
  - resources/**/*.yml
```

This tells the bundle: include any YAML file in resources, and in any sub-folder of resources. The job code is now hidden from databricks.yml but still deployed.

9.3 Use relative paths (very important)

The pasted YAML contains the notebook path of your own workspace, including your email. Another environment (QA, PROD) is a different workspace and a different user, so that path would not exist. Use a path **relative to the YAML file**.

Notation	Meaning
./	Same folder as the file
../	One folder up
../../	Two folders up

The file is at resources/jobs/demo_job.yml, so two levels up reaches the bundle root, then into source, notebooks, notebook_1. The task section becomes:

```
resources:
  jobs:
    demo_job:
      name: demo_job
      tasks:
        - task_key: ingest
          notebook_task:
            notebook_path: ../../source/notebooks/notebook_1.ipynb
```

> **Note:** Your pasted YAML will contain extra lines such as the environment settings. Keep those and change only the path. The snippet above shows the key part.

> **Why it matters:** Relative paths make the same YAML work in every environment, because the bundle looks inside itself, not at a hard-coded workspace location. Say this in interviews.

9.4 Deploy and fix the first error

Step 1. Run the deploy command:

```
databricks bundle deploy -t dev
```

Step 2. You may see an error such as: *notebook source/notebooks/notebook_1 not found*. The file is called notebook_1.ipynb, but the path lacked the extension, so it treated it like a folder.

Step 3. Add **.ipynb** at the end of the path, save, and deploy again.

Step 4. This time the CLI says uploading, deploying resources, updating deployment state, and finishes.

9.5 Verify

- In Workspace, the dev folder now has a **resources** folder with jobs and demo_job.yml.
- In **Jobs and Pipelines** you see two jobs: the one you made manually, and a new one created by the bundle with a prefix such as **[dev your_name] demo_job**.
- The bundle job shows a banner saying it was deployed as part of a bundle and that you should edit the source rather than this copy.

> **Tip:** Checkpoint: you can see both jobs and understand the bundle one has your name prefix. The next part explains why.

Part 10: Presets - Prefixes and Source-Linked Deployment

10.1 Why does the job name have a prefix?

In development mode, Databricks asset bundles apply default **presets**. The default name prefix looks like **dev + your username**. The reason: imagine two developers deploy the same bundle with a job called demo job. Without a prefix their names would conflict. With the prefix each developer gets their own copy and there is no conflict. Databricks adds it by default because you might forget.

Presets mostly matter for DEV. For QA and PROD they are optional.

10.2 The default formula

The default prefix is built from the current user: **dev** followed by the **short name** of the current user. In the bundle language this is the variable **${workspace.current_user.short_name}**.

10.3 Add your own custom presets

Under the dev target, add a presets block:

```
targets:
  dev:
    mode: development
    default: true
    presets:
      name_prefix: 'dev_${workspace.current_user.short_name}_'
      source_linked_deployment: false
    workspace:
      host: https://<your-workspace-url>
```

- **name_prefix**: any text you want, for example testing or a fixed word. It is added in front of every resource name. Keep it dynamic as shown instead of hard-coding a single person's name.
- **source_linked_deployment**: make it **false**. Explained next.

> **Warning:** Hard-coding a fixed prefix for everyone removes the protection that stops developers from clashing. That is why the video says you ideally should not do it. Keep the dynamic username in the prefix.

10.4 What is source_linked_deployment?

When you created the job, you picked a notebook that lived in the .bundle folder. With source-linked deployment turned on, the deployed job can keep referencing the source files at their original workspace location. When you change a notebook path in the YAML, you may expect the job to use the notebook inside the deployed bundle location, but it will not. Setting it to false forces the resources to rely on the files deployed with the bundle. You are overriding the default behaviour.

Step 1. Add the presets shown above and save.

Step 2. Run the deploy command again.

```
databricks bundle deploy -t dev
```

Step 3. In Jobs and Pipelines, the job now has the new prefix name. The previous prefixed job is no longer there because the name changed.

> **Tip:** Checkpoint: the job in Databricks now shows your custom prefix and it uses the notebook from the bundle files folder.

Part 11: How Bundles Track Changes (.databricks)

This part is advanced and rarely explained. Knowing it makes you stand out in interviews.

11.1 The sync snapshot

The hidden **.databricks** folder is the transaction log or version-control record of the bundle, similar to the delta log in Delta Lake and the .git folder in Git.

Step 1. In VS Code open **.databricks, bundle, dev** and then **sync-snapshots**.

Step 2. Open the JSON file. It is the snapshot of the **last deployment**: the files and their last-modified times.

Before every deployment the CLI compares your local files against this snapshot. If nothing changed since the last deployment it deploys nothing. If a file changed or a new one exists, it uploads only those files. This is an **incremental deployment**.

11.2 Prove it: add and change notebooks

Step 1. Create **notebook_2.ipynb** in source/notebooks with a print statement.

Step 2. Edit notebook_1 (for example print 'hello bro') and save.

Step 3. Run **databricks bundle deploy -t dev**.

Step 4. Open the snapshot JSON again. Only notebook_1 and notebook_2 have new timestamps. Everything else keeps old timestamps. Only the changes were deployed.

Step 5. Refresh the workspace. Both notebooks are in the notebooks folder.

11.3 The trap: deleting something directly in the workspace

Step 1. Delete **notebook_1** directly from the workspace (not from your project).

Step 2. Deploy again from VS Code.

Step 3. Refresh the workspace. notebook_1 is **still missing**.

Why? The CLI only checks its snapshot. The snapshot says notebook_1 was already deployed and is unchanged, so there is nothing to upload. It does not know you deleted it in the workspace. You are human and mistakes happen, so know the fix.

11.4 The fix

Remove the history so the bundle thinks everything is new, then deploy again.

Step 1. Delete the **.databricks** folder locally. (Deleting only the one snapshot entry caused a warning about a missing last-modified time, because the file existed before. With thousands of lines of code that can get messy.)

Step 2. Run **databricks bundle deploy -t dev** again.

Step 3. Output shows deploying and deployment complete. Refresh the workspace and notebook_1 is back. Jobs are also present with the dev prefix.

Tip: Recommended approach: delete the whole .databricks folder, remove history, and deploy fresh. Clean, simple, and safe.

Interview: Interview angle: how does a bundle know what changed? It compares the local files with the snapshot in .databricks stored per target, and deploys only changes.

Part 12: Deploy to PROD

So far everything went to DEV. Now add a production target.

12.1 Add the prod target

Both targets use the same workspace host in this demo because Free Edition gives you one workspace. In a real project the PROD host is a different workspace URL. You would create a second profile with the same login steps.

```
prod:
  mode: production
  workspace:
    host: https://<your-workspace-url>
    root_path: /Workspace/Users/<id>/prod/.bundle/${bundle.name}/${bundle.target}
  permissions:
    - group_name: admins
      level: CAN_MANAGE
```

- **mode: production**: production-mode behaviour (no dev prefix).
- **permissions**: you do not want everyone to change production resources. You can give a user_name, but in real work use a **group_name**. Here the admins group can manage the resource. If a group does not exist, create it under Settings, Identity and access, Groups.
- **root_path**: changes where the bundle is deployed. Here it deploys inside the prod folder you created, instead of your Home .bundle folder. Replace the user id with your own workspace user path.

> **Note:** The video typed the root path so the deployment lands in your workspace prod folder. If your user or folder differs, adapt the path. The pattern ends with the bundle name and the target.

12.2 Why you did not need a profile in bundle commands

When a target defines a host, the bundle looks for that host inside your .databrickscfg profiles. If it is found and already authenticated, all requests are authenticated automatically. You can still add the profile flag, but it is not required.

12.3 Validate and summarise before deploying

Critical environments deserve checks first.

Step 1. Remove the DEV job you created manually in the UI so names do not clash.

Step 2. Preview what will be deployed:

```
databricks bundle summary -t prod
```

Step 3. It shows the host, the user deploying, the workspace path, and the resources (here, one job).

Step 4. Run validation:

```
databricks bundle validate -t prod
```

Step 5. You may see a warning. As long as validation reports success, continue. Warnings here are fine in the demo.

12.4 Deploy to PROD

```
databricks bundle deploy -t prod
```

On Free Edition you may see a permissions error about applying permissions. That is expected because Free Edition does not allow full permissions. The resources can still deploy.

12.5 Verify

- In Jobs and Pipelines there is one job deployed by the bundle. Open it: the task says it was deployed as part of a bundle.
- In Workspace open the **prod** folder, then .bundle, dab_tutorial, prod. You find source, resources, fixtures and both notebooks.
- Open the **.databricks** folder locally. There are separate snapshots for **dev** and **prod**.

> **Why it matters:** Each target has its own snapshot. That is why a deployment to dev never confuses the state of prod.

> **Tip:** Checkpoint: you can deploy the same bundle to dev and prod with one flag change.

Part 13: Git Folders and the Web Terminal

Now you do the same work inside Databricks, with no local VS Code. Many large organisations prefer this because a local terminal can run anything, which is a risk. Using a cloud Web Terminal keeps control in the workspace. You will use this approach for the remainder of the guide.

13.1 Background

Databricks Git folders were previously called Databricks Repos. A Git folder lets you work on branches in isolation so you do not overlap with other people. Basic Git knowledge helps (branch, commit, push, pull request).

13.2 Create a GitHub repository

Step 1. Sign in to GitHub and click **New repository**.

Step 2. Name it **asset_bundles**, choose private or public, tick a README so it is not empty, and create it.

13.3 Link GitHub to Databricks

Step 1. In Databricks click your profile icon, then **Settings**, then **Linked accounts**.

Step 2. Click **Add Git credential**, set the nickname to **GitHub**.

Step 3. Choose **Link Git account** (preferred over a personal access token, which was the older method).

Step 4. Authorize the connection. It shows link success.

13.4 Create the Git folder

Step 1. In the repository on GitHub click **Code** and copy the HTTPS clone URL.

Step 2. In Databricks go to **Workspace, Home**. Click **Create, Git folder**.

Step 3. Paste the URL. The Git provider and folder name fill in automatically. Keep the name as is.

Step 4. Click **Create Git folder**. You see a folder with the repo icon, the main branch, and the README.

13.5 Create a feature branch

Never develop directly in main.

Step 1. Click the branch button at the top of the Git folder.

Step 2. Create a new branch, for example **feature_branch**, and click **Create**.

> **Warning:** First-time error: if branch creation fails, the message includes a link. Click the first link and install the Databricks application in your GitHub account (one-time task). Then try again.

13.6 Open the Web Terminal

A terminal needs a cluster, so you attach compute to a notebook first.

Step 1. Inside the Git folder click **Create, Notebook**. Name it **demo**. Attach compute (serverless works).

Step 2. Open the terminal in one of two ways: the compute dropdown, then **Web terminal**; or the terminal icon at the bottom of the notebook.

Step 3. Wait a few seconds. You now have a terminal similar to VS Code. It can be opened in its own window for more room.

> **Note:** The Web Terminal session ends when the terminal closes or compute times out. If you see Connection closed, simply open a notebook, attach compute again, and reopen the terminal.

Part 14: Initialise a Bundle in the Git Folder

14.1 Move to the right directory

The terminal opens in a default location. You must change into your Git folder before initialising.

Step 1. Run cd and give the full workspace path of your Git folder. Example:

```
cd /Workspace/Users/<your-email>/asset_bundles
```

Step 2. Initialise:

```
databricks bundle init
```

Step 3. Choose the default template and use the project name **dab_project**. Answer yes to the sample options.

Step 4. Move into the project:

```
cd dab_project
```

> **Note:** You do not need a .databrickscfg file here. Authentication is handled by the Databricks workspace you are already logged into.

14.2 Clean and configure

Step 1. In the Git folder you now have the dab_project folder. Delete the sample content in resources, source and tests exactly as in Part 7.

Step 2. Open **databricks.yml** (the file banner may mention switching to the associated asset bundle - that is fine).

Step 3. Delete the artifacts section.

Step 4. Set the real host. Get it from Compute, SQL Warehouses, Connection details, or copy the workspace URL with https://.

> **Warning:** All of these changes happen in the **feature branch**, not main. Check the branch name at the top of the Git folder.

Part 15: Variables and Dynamic Deployment

Real projects have a different catalog per environment: a dev catalog, a QA catalog, a prod catalog. You do not want to hard-code catalog names. Variables make the deployment dynamic. CI/CD is incomplete without them.

15.1 Write a parameterised notebook

Step 1. Inside **source** create a folder **notebooks**.

Step 2. Create a notebook named **ingestion**.

Step 3. First test a query with a hard-coded catalog to see it works. The video queries the information schema columns table of the asset_bundles catalog.

Step 4. Make the catalog a widget parameter, then use it in an f-string. Final notebook code:

```
dbutils.widgets.text("catalog_name", "asset_bundles")
catalog_name = dbutils.widgets.get("catalog_name")

df = spark.sql(f"SELECT * FROM {catalog_name}.information_schema.columns")
display(df)
```

Whatever value arrives as the parameter is used as the catalog. For example a QA catalog name or a prod catalog name.

15.2 Create the job in the UI

Step 1. Go to **Jobs and Pipelines, Create, Job**.

Step 2. Task type Notebook. Choose the ingestion notebook from your Git folder path (users, your user, asset_bundles, dab_project, source, notebooks, ingestion).

Step 3. Add a task **parameter**: key **catalog_name**, value **asset_bundles** (it will be replaced by a variable in the bundle).

Step 4. Create the task. Rename the job to **ingestion_job**.

This job is not yet deployable as is, because the value is hard-coded. It would run in dev, but QA and PROD use different catalogs.

15.3 Copy the YAML into a resource file

Step 1. Three dots menu, **Edit as YAML**, and copy it.

Step 2. In the bundle create **resources/jobs/job.yml** (the video also names it job.yml) and paste.

Step 3. Change the notebook path to a relative path (notebook files in the Git folder usually have no extension; use whichever the file name shows):

```
notebook_path: ../../source/notebooks/ingestion
```

If you created a .ipynb file instead, add the extension exactly as in Part 9.

15.4 Create the variable

The best practice from the video: it is more readable to define variables in databricks.yml. Some people keep them as separate files under resources. Both work. This guide follows the separate-file approach shown on screen, since include already loads everything under resources.

Step 1. Create the folder **resources/variables**.

Step 2. Create **variables.yml** (or environments.yml).

Step 3. Define the variable with an optional description and a **default** value. Set the default to your DEV value in case you forget to supply one:

```
variables:
  catalog_name:
    description: Catalog used by jobs and pipelines
    default: asset_bundles
```

15.5 Refer to the variable

Use the dollar sign, curly braces, **var** dot the variable name:

```
base_parameters:
  catalog_name: ${var.catalog_name}
```

> **Warning:** The prefix is var. In the video the first attempt used the wrong word (the speech rendered it as where) and deployment failed with a message that a variable was declared in the root. Always write ${var.catalog_name}. If the variable name in your bundle does not exist, the deployment fails.

15.6 Make sure include is present

In databricks.yml:

```
include:
  - resources/*.yml
  - resources/**/*.yml
```

This loads the job file and the variables file.

Part 16: Add a Lakeflow (Delta Live Tables) Pipeline

Delta Live Tables are now named Lakeflow Declarative Pipelines. This guide does not teach the full pipeline topic. It teaches how to **include a pipeline in a bundle**, including the new pipeline editor, which is tricky to configure.

16.1 Turn on the new editor

Step 1. Click your profile icon, then **Settings**, then **Developer**.

Step 2. Enable the **Lakeflow pipeline editor** setting. Also enable the setting for tabs for notebooks and files.

16.2 Create a demo pipeline

Step 1. Go to **Jobs and Pipelines, Create, ETL pipeline**.

Step 2. Name: **demo_pipeline**.

Step 3. Catalog: **asset_bundles**. Schema: **bronze** (any schema is fine, it will be dynamic later).

Step 4. Choose **Python** to start.

Step 5. Remove the extra sample items so you have a clean structure.

Step 6. Add a file **transformation.py** under the transformations folder with this basic code:

```
import dlt

@dlt.table
def transformed_data():
    return spark.range(10)
```

> **Note:** Newer pipelines may use the pyspark pipelines module instead of dlt, but the video uses the dlt module. Use what your editor generates.

Step 7. Click **Run pipeline**, then **Dry run**. Always dry run first. It validates without running.

16.3 Move the pipeline root into your bundle

A pipeline created from the UI lives outside your bundle by default (under your user folder). You must move its whole root inside your bundle source folder so the bundle can deploy it.

Step 1. In the pipeline editor open the settings and change the **root folder**.

Step 2. Create a folder **pipelines** under source in your bundle (users, your user, asset_bundles, dab_project, source, pipelines).

Step 3. Move the demo pipeline folder there. The structure becomes source/pipelines/demo_pipeline with the transformations folder inside.

16.4 Copy the pipeline YAML

Step 1. Go to Jobs and Pipelines, open **demo_pipeline**, and open **Settings**.

Step 2. Click **View settings YAML** and copy it.

Step 3. In the bundle create **resources/pipelines/demo_pipeline.yml** and paste.

16.5 Fix the paths and the catalog

Change the root path and the transformations library path to relative paths, and replace the hard-coded catalog with the variable:

```
resources:
  pipelines:
    demo_pipeline:
      name: demo_pipeline
      catalog: ${var.catalog_name}
      schema: bronze
      serverless: true
      root_path: ../../source/pipelines/demo_pipeline
      libraries:
        - glob:
            include: ../../source/pipelines/demo_pipeline/transformations/**
```

- **root_path** is the root of the whole pipeline project, not only the transformation files.
- Transformations are whatever lives in the transformations folder. If you rename that folder (for example to enriched), update the library include path too.
- Both are two levels up from resources/pipelines to the bundle root, then down into source.

16.6 Set the targets with presets

Make sure databricks.yml has the dev target with presets and the prod target. The same as before:

```
targets:
  dev:
    mode: development
    default: true
    presets:
      name_prefix: 'dev_${workspace.current_user.short_name}_'
      source_linked_deployment: false
    workspace:
      host: https://<your-workspace-url>

  prod:
    mode: production
    workspace:
      host: https://<your-workspace-url>
      root_path: /Workspace/Users/<id>/prod/.bundle/${bundle.name}/${bundle.target}
    permissions:
      - group_name: admins
        level: CAN_MANAGE
```

16.7 Validate, summarise and deploy to DEV (Web Terminal)

Step 1. If the terminal closed, open any notebook, attach compute, and reopen the terminal in a new window. (The video attached an environment version 3 compute. Use whatever compute Databricks offers.)

Step 2. Go into the project folder:

```
cd /Workspace/Users/<your-email>/asset_bundles/dab_project
```

Step 3. Validate the dev target:

```
databricks bundle validate -t dev
```

Step 4. It prints the bundle name, target, host, user and path, then reports validation OK.

Step 5. See the summary:

```
databricks bundle summary -t dev
```

Step 6. You should see **two resources**: one job and one pipeline.

Step 7. Deploy:

```
databricks bundle deploy -t dev
```

Step 8. If you get an error about the variable (the earlier var typo), fix it and run deploy again. A partially failed deployment is safe to retry.

16.8 Verify

- Jobs and Pipelines now shows both the job and the pipeline, prefixed for dev.
- Open the job, three dots, Edit as YAML: the catalog name is the default value (asset_bundles). You did not hard-code it, the default came from the variable.
- Open the pipeline settings YAML: it uses the bundle location because source_linked_deployment is false.
- In the workspace .bundle folder you see dab_project and the dev files.

> **Tip:** Checkpoint: both resources are deployed to dev and use the variable default.

Part 17: Override Variables at Deploy Time

The default value is for DEV. For PROD you want a different catalog. You can override a variable during deployment with the **--var** flag, without touching any YAML.

17.1 Find the flag

```
databricks bundle deploy -h
```

In the global flags you see an option to set values for variables defined in the bundle config. The example shows name equals value format. (In the video's transcript the flag is spoken in a way that is mistaken for where; the real flag is **--var**.)

17.2 Deploy to PROD with an overridden catalog

Step 1. Delete any earlier DEV job and pipeline if they would clash in names.

Step 2. Run:

```
databricks bundle deploy -t prod --var="catalog_name=asset_bundles_prod"
```

Step 3. The first time you may see an error: *cannot create pipeline, catalog asset_bundles_prod does not exist*. That proves the override is working. Fix it by creating the catalog.

Create the PROD catalog

Step 4. Go to Catalog, click Create catalog, name it **asset_bundles_prod**, save.

Step 5. If the pipeline needs the bronze schema, create it as in Part 5.

Step 6. Run the same deploy command again. It completes. A permissions error on Free Edition is fine.

17.3 Verify PROD

- Two production resources are deployed: demo_pipeline and ingestion_job (no dev prefix).
- Open the job YAML: the catalog name is **asset_bundles_prod** - overridden.
- Open the pipeline settings YAML: it also shows the overridden catalog.
- In Workspace, prod, .bundle, dab_project, prod, you find files, source and all folders.
- The job and pipeline reference the notebook and code inside the prod bundle location, not your development path, because source_linked_deployment was set to false.

> **Interview:** Interview angle: how do you deploy the same bundle to different catalogs? Define a variable with a default for DEV, and override it with --var for QA and PROD (or set values per target).

Part 18: Destroy a Bundle

Sometimes you want to remove everything a bundle deployed.

18.1 What destroy does

- Deletes all the resources the bundle deployed (jobs and pipelines).
- Deletes the deployed files in the bundle folder. The .bundle folder itself remains but becomes empty.
- It cannot be undone. Your source code still exists in Git if you committed it.

18.2 Run it

Step 1. Read the help first:

```
databricks bundle destroy -h
```

Step 2. Destroy the PROD deployment:

```
databricks bundle destroy -t prod
```

Step 3. A warning lists what will be deleted and asks for confirmation. Confirm. The output says deleting files and destroy complete.

Step 4. Check Workspace, prod folder: the .bundle folder is empty. Check Jobs and Pipelines: both production resources are gone.

> **Warning:** Use the target flag carefully. Destroying the wrong target removes the wrong environment.

18.3 Why destroy is useful

It is a clean-up tool for test environments, abandoned features, or resetting a target before redeploying from scratch.

Part 19: Push Changes to GitHub and Merge

Your bundle work should be saved in Git and shared. This part completes the Git workflow in Databricks.

Step 1. Open the Git folder and the branch dialog. You are on **feature_branch** with many changes.

Step 2. Enter a commit message such as **bundle development**.

Step 3. Click **Commit and push**. This commits and pushes the branch to the remote GitHub repo.

Step 4. On GitHub, refresh. You see the feature branch with a recent push and a **Compare and pull request** prompt. Main only has the README at this point.

Step 5. Click **Compare and pull request**, then **Create pull request**. In a real team someone reviews and approves. In this demo you simply merge.

Step 6. Click **Merge pull request** and confirm.

Step 7. Main on GitHub now contains the dab_project bundle. You can showcase it on your profile.

Step 8. Back in Databricks, switch to the **main** branch. Your local main is still behind the remote main. Click **Pull**, confirm any warning (it may mention clearing outputs), and the files appear.

> **Tip:** If you do not see the new files in your local main, you only need to pull. Pull brings in changes that exist in the remote main and not in the local main.

Part 20: Command Cheat Sheet

Task	Command
Install CLI (Windows)	winget install Databricks.DatabricksCLI
Check version	databricks -v
Install uv	pip install uv
Log in (create profile)	databricks auth login --host https://<url>
List profiles	databricks auth profiles
Help for any command	databricks <command> -h
List catalogs	databricks catalogs list --profile dev
Get a catalog	databricks catalogs get <name> --profile dev
Create a schema	databricks schemas create <schema> <catalog> --profile dev
Initialise a bundle	databricks bundle init
Validate	databricks bundle validate -t <target>
Summary	databricks bundle summary -t <target>
Deploy	databricks bundle deploy -t <target>
Deploy with override	databricks bundle deploy -t prod --var="name=value"
Destroy	databricks bundle destroy -t <target>

Variable and path syntax

Syntax	Use
${var.name}	Refer to a custom variable you defined
${workspace.current_user.short_name}	The current user's short name (used in the dev prefix)
${bundle.name}, ${bundle.target}	Bundle and target names, useful in root_path
../../source/...	Relative path from a resource YAML file to the source folder

Part 21: Troubleshooting Guide

Problem	Cause	Fix
databricks is not recognised	PATH not refreshed after winget install	Close all terminals and VS Code, wait 5-10 min, restart the laptop if needed.
Catalog create fails with metastore root URL error	Free Edition does not give metastore control	Create the catalog in the Catalog UI instead.
Errors about the host when deploying	Host copied without https://	Always use the full https:// URL in the profile and databricks.yml.
Notebook not found when deploying a job	Path missing the .ipynb extension, so it is treated as a folder	Add the extension to notebook_path.
Resources folder missing in the workspace	Folder was empty	Add at least one resource file; empty folders are not deployed.
Deleted file does not come back after deploy	Snapshot says it is unchanged	Delete the local .databricks folder and deploy again.
Variable declared in the root error	Used the wrong prefix instead of var	Use ${var.catalog_name} and make sure the variable exists.
Cannot create pipeline, catalog does not exist	The overridden catalog is not created yet	Create the catalog (and schema) and deploy again.
Permissions error on deploy (Free Edition)	Free Edition limits permission settings	Expected. Resources can still deploy.
Branch creation fails in a Git folder	Databricks GitHub app not installed	Open the link in the error and install the app once.
Web terminal says Connection closed	Compute or session ended	Open a notebook, attach compute, reopen the terminal.
Local main shows old code	Remote has updates not pulled	Switch to main and click Pull.
Two people overwrite each other's jobs in DEV	No unique prefix	Keep the dev name_prefix with the user short name.

Part 22: Interview Questions and Answers

Use these to practise. The answers match what the video teaches, so you can say them with confidence.

Q1. What are Databricks Asset Bundles?

A way to define a Databricks project end to end as code: source files plus resources such as jobs and pipelines, and the targets they deploy to. A bundle is deployed as a single unit to environments like dev and prod.

Q2. Where do bundles fit in CI/CD?

On the deployment (CD) side. Bundles make deployments to QA and PROD repeatable and configurable.

Q3. What is the backbone of asset bundles?

The Databricks CLI. Bundle commands such as init, validate, deploy and destroy are built on top of it, and the CLI itself wraps the REST APIs.

Q4. How do you authenticate the CLI?

With U2M (user to machine) through databricks auth login, which creates a profile in .databrickscfg. M2M with service principals is used for automation.

Q5. What is the .databrickscfg file and DEFAULT section?

The local config file holding profiles. DEFAULT is a fallback profile used when no profile is specified.

Q6. What is databricks.yml?

The main bundle file: bundle name, includes, variables and targets with workspace host, mode, presets and permissions.

Q7. Why keep jobs in separate YAML files?

Readability. databricks.yml stays focused on targets and settings, and include pulls in the resource files.

Q8. Why use relative paths for notebooks?

Because the same bundle deploys to several workspaces. Absolute user paths do not exist elsewhere.

Q9. What are presets?

Settings that customise target behaviour. In dev, a default name prefix of dev plus the username prevents name clashes between developers. Another preset is source_linked_deployment.

Q10. Why set source_linked_deployment to false?

So the deployed resources rely on the files inside the bundle deployment, not on the original linked source location.

Q11. What is the .databricks folder?

The transaction log of the bundle. It holds sync snapshots per target so the CLI deploys only changes.

Q12. You deleted a notebook in the workspace but redeploy does not bring it back. Why, and how do you fix it?

The snapshot says it is unchanged, so nothing is uploaded. Delete the .databricks folder (remove history) and deploy again.

Q13. How do you make deployments dynamic across environments?

Define variables with defaults and reference them as ${var.name}. Override at deploy time with --var, or per target.

Q14. What do validate and summary do?

Validate checks the bundle config for a target. Summary lists the resources and paths that would be deployed. Use both before deploying to PROD.

Q15. How do you control who can manage production resources?

Using permissions in the prod target, preferably a group name with a level such as CAN_MANAGE.

Q16. What does bundle destroy do?

Deletes all resources deployed by the bundle for that target and removes the deployed files. It cannot be reverted.

Q17. How do you work with bundles without a local terminal?

Use Databricks Git folders and the Web Terminal, with the CLI authenticated through the workspace.

Q18. What is special about including pipelines with the new editor?

You must move the pipeline root into the bundle source and set root_path and the libraries glob to relative paths.

Q19. Who creates bundles in a team?

The data engineer authors and tests them. DevOps then automates deployment to QA and PROD.

Part 23: Final Checklist and Key Takeaways

23.1 Build checklist

Done	Item
[ ]	Free Edition account created
[ ]	VS Code, Python, uv installed
[ ]	Databricks CLI installed and databricks -v works
[ ]	dev profile created with databricks auth login
[ ]	Catalog asset_bundles and schema bronze created
[ ]	Bundle initialised (dab_tutorial) and cleaned
[ ]	Notebook deployed to dev
[ ]	Job added with relative paths and deployed
[ ]	Presets added (name_prefix, source_linked_deployment false)
[ ]	Change tracking understood and .databricks reset tested
[ ]	PROD target validated, summarised, deployed
[ ]	GitHub repo, linked account, Git folder, feature branch
[ ]	Web terminal bundle (dab_project) with variables, job, pipeline
[ ]	Variable overridden with --var for PROD
[ ]	Bundle destroyed and changes pushed, merged, pulled

23.2 Ten things to remember

- The CLI is the backbone. Bundles are built on it.
- A bundle is an end-to-end project definition as code.
- Use the UI to build a job or pipeline, then copy its YAML.
- Keep resources in separate files and load them with include.
- Always use relative paths so one bundle works in every environment.
- In dev, names get a username prefix so developers do not clash.
- Set source_linked_deployment to false so the bundle's own files are used.
- The .databricks folder is the transaction log, with a snapshot per target.

- Use variables with ${var.name} and override with --var.
- Always validate and summarise before PROD. Destroy cannot be undone.

> **Tip:** Where to go next: the video author suggests his Lakeflow Jobs course, his Declarative Pipelines course and a full Git course for data engineers. They deepen the topics you used here.